# Parallel Scientific Computing: GPU & MPI

Toby Davis (cltz62)

*Durham University*

07 April 2026

## I. Hardware & System

All benchmarks were performed on Durham University's NCC cluster with a single CPU core and an NVIDIA 2080 Ti GPU with 11GB of GDDR6 VRAM and compute capability 7.5. The 2080 Ti GPU has a peak memory bandwidth of 616GBs$^{-1}$ and can perform 13.4TFLOPs$^{-1}$ in FP32 or 107TFLOPs$^{-1}$ in FP16. All benchmarks were compiled using CUDA 12.4 and Intel OneAPI MPI 2024.1.0.

## II. Vector Operations

### 1. Vector Triad

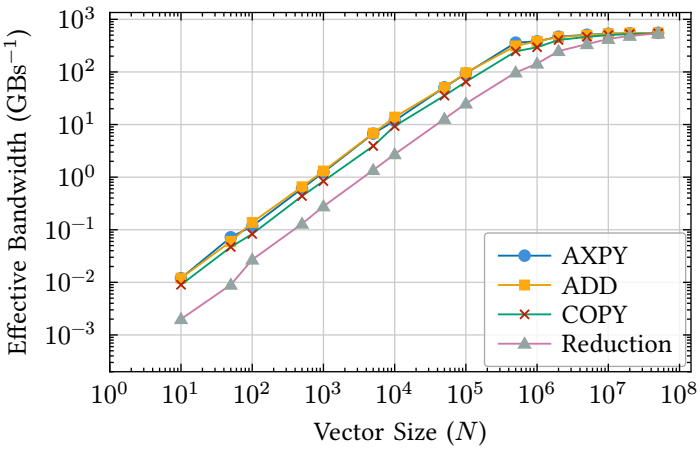


Figure 1: Effective memory bandwidth for vector operations at varying problem sizes with a single MPI rank.

The vector triad operations shown in Figure 1 all show very similar memory bandwidth characteristics across different problem sizes. For small $N$, there is not enough data to fully saturate the GPU memory bus and CUDA cores, so memory and kernel dispatch latencies dominate and bandwidth is low. As the problem size exceeds the number of CUDA cores on the GPU, memory transfers can be overlapped with compute and memory bandwidth becomes the bottleneck.

The 2080 Ti GPU has a maximum memory throughput of 616GBs$^{-1}$ and the highest recorded throughput in the vector triad operations was 553.3GBs$^{-1}$, suggesting that the kernels were not able to fully saturate the memory bus, even at large problem sizes. Effective bandwidth is computed as the total bytes transferred divided by the kernel time. AXPY and ADD each read two arrays and write one ($3N \times 4$ bytes), COPY reads one and writes one ($2N \times 4$ bytes), and the reduction reads a single array ($N \times 4$ bytes).

Figure 2 shows the total bandwidth across all MPI ranks for a fixed problem size and varying numbers of MPI ranks. Increasing the number of MPI ranks reduces the amount of data each rank must process and reduces its effective bandwidth, as shown in Figure 1. While some of this computation can be overlapped, the overhead of doing so reduces the available parallelism and leads to degradation in performance.

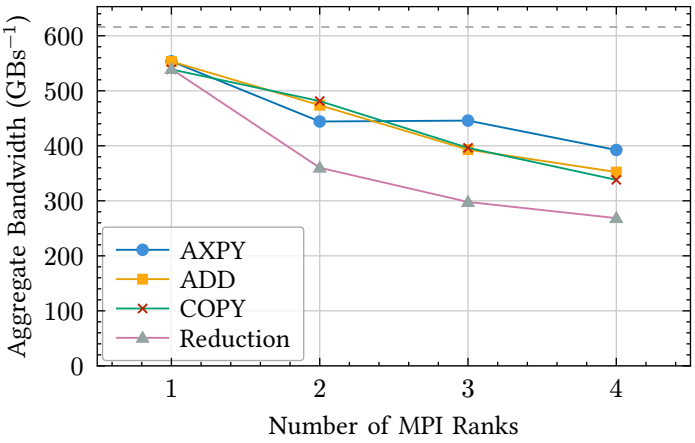


Figure 2: Aggregate memory bandwidth across MPI ranks for $N = 50 \times 10^6$. Dashed line shows 2080 Ti peak (616GBs$^{-1}$).

### 2. Reduction

We implemented a tree-reduction algorithm with warp-level primitives to reduce synchronisation overhead. We spawn at most 256 blocks, each containing 1024 threads. Each thread reduces a section of the problem into a local sum, which is reduced via a warp-level reduction into another partial sum. Another pass of the reduction algorithm on the partial sums is enough to compute the final result. We load data as `float4` arrays to maximise memory bandwidth and ensure that all threads in a warp are active to achieve the best possible performance, reaching 546.1GBs$^{-1}$, matching the vector triad bandwidth.

IEEE 754 32-bit floating point numbers have a 23-bit mantissa, allowing for relative error on the order of $\frac{1}{2^{23}}$. Additionally, we can approximate the sum as a random walk of length $N$, so $\sum_i x_i \approx \sqrt{N}$. Hence, we can estimate the floating point error on a reduction of $N = 50 \times 10^6$ elements uniformly centred around 0 as

$$\frac{1}{2^{\text{mantissa}}} \times \sqrt{N} \approx 1.192 \times 10^{-7} \times 7071.067 \approx 0.00084.$$

The maximum error observed with our reduction algorithm is 0.000732422, measured by comparing the GPU result against a double-precision CPU reference sum across multiple random seeds. This confirms the algorithm is numerically stable for large $N$.

## III. General Matrix Multiply (GEMM)

In addition to the naive and tiled implementations, we implemented an optimised F16 tiled WMMA algorithm which multiplies in F16 and accumulates into F32. Figure 3 shows the performance of each GEMM algorithm over varying matrix sizes.

The naive implementation assigns one thread per output element, so each thread independently loads an entire row of $A$ and column of $B$ from global memory with no data reuse

between threads. This gives an arithmetic intensity of roughly one floating point operation per four bytes loaded. Once the input matrices exceed the L2 cache, every access goes to VRAM at full latency, reducing performance by almost a third from its peak. The maximum observed error was 0.000019. For small matrices the L2 cache is sufficient to keep the GPU fed and the naive implementation performs competitively with the tiled version.

The tiled implementation uses $32 \times 32$ tiles loaded into shared memory with `__syncthreads()` barriers between tile loads and computation to prevent data races. Each thread accumulates its output element across all tiles along the $K$ dimension, reusing data more efficiently and avoiding the penalties of fetching from VRAM. This reaches a peak throughput of 1556.5GFLOPs$^{-1}$ with a maximum error of 0.000019.

By efficiently tiling the matrices, using tensor cores and reducing the precision of the multiplications, the optimised algorithm is over 20 times faster than the tiled version and yet has an error of just 0.011714.

## 1. Problem Size Scaling

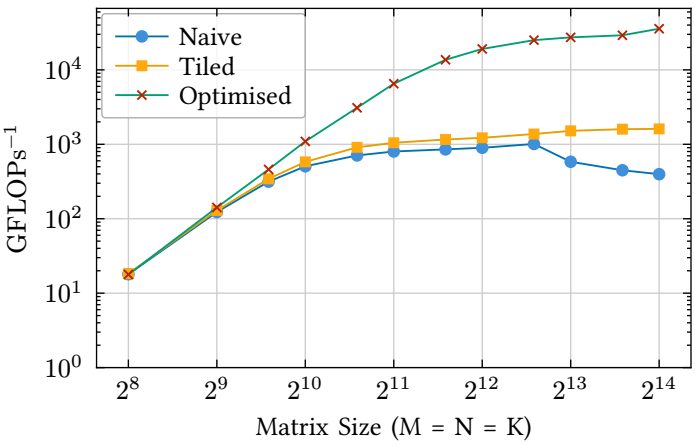


Figure 3: GEMM throughput (GFLOPs$^{-1}$) for each kernel at varying matrix sizes with a single MPI rank.

## 2. Strong Scaling

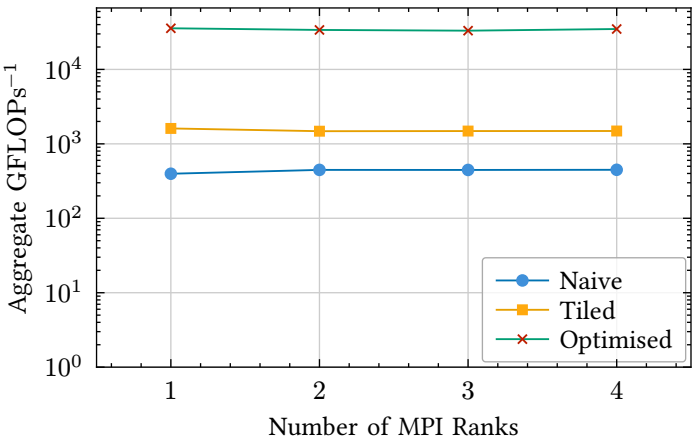


Figure 4: Aggregate GFLOPs$^{-1}$ for GEMM kernels as MPI ranks increase, showing scaling on a single GPU.

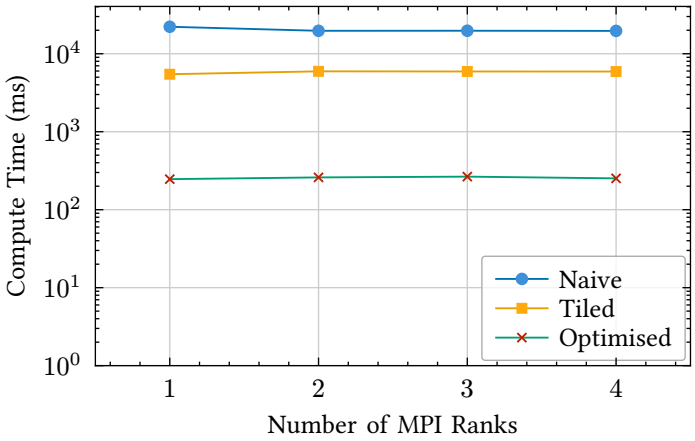


Figure 5: Rank 0 compute time for GEMM kernels as MPI ranks increase. Each rank computes fewer rows, but GPU contention limits speedup.

| Ranks | Compute (ms) | Comm (ms) | Total (ms) | Comm % |
|---|---|---|---|---|
| 1 | 246.434 | 0.003 | 246.437 | 0.001% |
| 2 | 259.419 | 0.287 | 259.706 | 0.111% |
| 3 | 265.406 | 0.016 | 265.422 | 0.006% |
| 4 | 252.119 | 0.017 | 252.136 | 0.007% |

Table 1: Compute and communication time for the optimised GEMM kernel at $M = N = K = 16384$.

For a fixed problem size of $M = N = K = 16384$, Figure 4 and Figure 5 show that the GPU throughput and compute time are not dependent on the number of MPI ranks used. This is because the GPU is fully occupied by the computation and there is no way to overlap the per-rank computation, so they run near-sequentially and the runtime is unaffected.

Communication time was measured separately via an MPI barrier before each kernel launch. In the worst case, communication jitter takes 26.5% of the total time. On average, however, communication accounts for only 7.93% of the total runtime. This suggests that MPI communication is not the bottleneck, and that a more optimised GEMM implementation would lead to consistently better performance.

The factor limiting the scaling of the GEMM kernel is the GPU's compute throughput and memory bandwidth. If distinct GPUs could be assigned per rank, the throughput could be expected to scale linearly with the number of ranks due to the almost negligible communication overhead.

# 3. Profiled Results

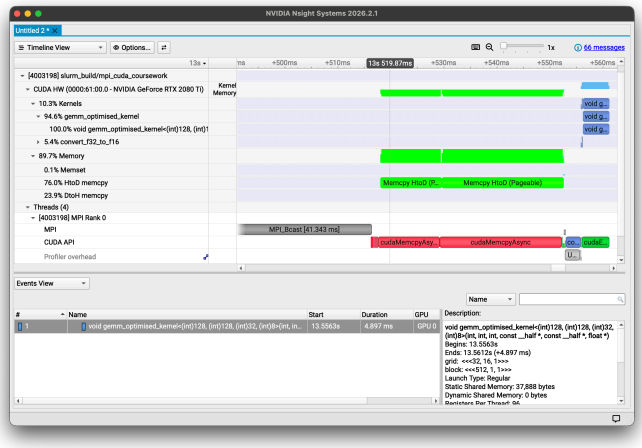


Figure 6: Profiling a 2-rank benchmark in Nsight Systems

The profiling results shown in Figure 6 report that only 5.4% of the time is spent in `convert_f32_to_f16`, with the remaining 94.6% of the time being spent in the WMMA GEMM kernel. It is clear that further optimisation efforts should be focused on the compute-intensive GEMM kernel rather than the simple and fast data cast/copy. The profile also shows that memory copies dominate the runtime of the program and therefore should be avoided at all costs.

The profile also shows that very minimal time is spent in MPI communication, which is consistent with the benchmark results above. GPU idle time between kernel launches is negligible, confirming that the device is compute-bound. Each rank launches both the F32-to-F16 conversion and the WMMA GEMM, with launches separated only by the implicit synchronisation between them.

Given more time, we would write inline PTX to better overlap memory loads with compute and to gain finer control over tile sizes. On newer GPU architectures (sm_80+), WMMA supports F32 operands, which would eliminate the precision penalty of F16 while retaining tensor core throughput.